

Code Arena v2.0 — Complete Production & Scalability Plan

A Master Architectural Roadmap for Enterprise Multi-Language Code Judging, Zero-Trust Sandboxing, AI Mentorship, and 10,000+ Scholar Scalability

1. Executive Summary

The current Education Algorithm Code Arena features a strong foundation: student authentication, CSRF security, rate limiting, challenge management, AI Code Mentor, and MySQL database integration. The primary objective of this production upgrade is to **replace untrusted host-level process execution with a zero-trust, isolated, distributed judging engine** while keeping the existing LMS website, user accounts, and UI 100% intact.

The recommended strategy is **incremental replacement, not a rewrite**. The main website handles user traffic and enqueues execution jobs into a Redis queue. Isolated judge worker nodes pull jobs, execute code inside micro-sandboxes, and return real-time verdicts.

2. Target System Architecture

The architecture strictly decouples the web server from code execution:

Layer	Technology Component	Responsibility & Function
Frontend UI	Monaco Editor + SSE / WebSockets	Code editing, real-time syntax checking, live iframe viewport, streaming execution status.
Edge API Gateway	PHP 8.3 FPM (api-code-runner.php)	Auth & CSRF verification, per-IP rate limiting, payload validation, job enqueueing to Redis.
Queue / Event Bus	Redis 7 Enterprise	Priority job queues, pub/sub channels for live verdict streaming, rate limit token storage.
Judge Worker Pool	Isolated Worker Nodes (cgroups v2)	Pulls jobs, compiles & executes code in tmpfs RAM Disk sandbox, tests public/hidden cases.
Database Tier	MySQL 8 Enterprise Cluster	Stores challenges, test cases, submission records, leaderboard points, MOSS signatures.

3. System Scope: What Stays vs. What Changes

What STAYS Intact (Zero Disruption):

- Existing Code Arena UI, Monaco editor, and student dashboard.
- Student login, session authentication, and database tables.
- CSRF protection, IP rate limiting, and course integrations.
- AI Code Mentor interface and CDN-based video delivery.

What CHANGES (Backend Upgrades):

- Move execution off main web server to isolated worker containers.
- Replace transformation/simulation with real compilers (javac, g++, python3, node).
- Introduce automated hidden test-case judging and special checkers.
- Enforce asynchronous Redis job queue for 1,000+ student concurrency.

4. Execution Service Design & Resource Quotas

Every code execution job runs inside an ephemeral, zero-trust sandbox utilizing Linux **cgroups v2**, **seccomp-bpf syscall filtering**, and **tmpfs RAM Disk** storage. Files are compiled and executed in-memory with zero physical disk I/O latency and zero persistent footprint.

Resource Parameter	Enforced Hard Limit	Security Mechanism
CPU Cores	1.0 Dedicated vCPU	cgroup CPU quota allocation
RAM Memory	256 MB	Out-Of-Memory (OOM) SIGKILL
Execution Timeout	2.000 Seconds	SIGXCPU / SIGKILL timer
Max Processes/Threads	10 Maximum	cgroup pids limit (prevents fork bombs)
Network Access	DISABLED (Network None)	Isolated network namespace / no interface
Output Size Cap	1.0 MB Maximum	Process stdout/stderr truncation (SIGXFSZ)
Filesystem	tmpfs (In-Memory RAM)	Read-only rootfs + temporary RAM disk

5. Supported Multi-Language Matrix

Language	Toolchain / Compiler	Execution Environment & Optimizations
Java 21 LTS	OpenJDK 21 (javac / java)	Real bytecode compilation (-Xms32m -Xmx256m -XX:+UseG1GC).
Python 3.12	Python 3.12 & PyPy3 JIT	Direct bytecode execution with -O flag, memory limit guard.
C++20	GCC 13.2 (g++) / Clang 17	Optimized binary (-O3 -std=c++20) + AddressSanitizer (ASan).
JavaScript / TS	Node.js 20 LTS / Bun	V8 Engine with --max-old-space-size=256 RAM cap.
HTML5 / CSS3	Live Web Viewport	Client-side isolated HTML5 iframe preview for Web GUI exercises.
SQL Arena	In-Memory SQLite PDO	Seeded mock enterprise schema (users, departments, salaries).

6. Deep-Dive: Execution Pipeline (api-code-runner.php)

The backend execution script follows a 6-stage security pipeline:

- 1. Non-Blocking Session Release:** Calls session_write_close() immediately so parallel AJAX requests and editor tabs never block the student browser.
- 2. CSRF & Rate Limit Guard:** Verifies CSRF token hash and applies 60 runs/min sliding-window rate limit per student/IP.
- 3. AST Security Linter (Pre-Flight):** Scans source code against regex patterns blocking system calls (os.system, subprocess, Runtime.getRuntime(), child_process, raw sockets).
- 4. Language Routing Sandbox:** Routes code to specific compilers/interpreters (Java, Python, C++, Node.js, SQLite, or HTML Iframe viewport).
- 5. Test Case Evaluation:** Normalizes output (strips trailing whitespace/newlines) and compares student output against public and hidden test cases.
- 6. MOSS Plagiarism Check & DB Insert:** Calculates token stream similarity and stores submission record in MySQL code_submissions.

7. AI Code Mentor 2.0 & Anti-Cheating Suite

5-Tier Progressive Hint Engine (Prevents direct copy-pasting):

- **Hint 1 (Concept):** Explains core problem concepts and inputs without giving code.
- **Hint 2 (Pattern):** Identifies key data structures/patterns (Two Pointers, Sliding Window, Monotonic Stack).
- **Hint 3 (Algorithm):** Outlines step-by-step logic in natural language.
- **Hint 4 (Pseudocode):** Provides high-level pseudocode structure.
- **Hint 5 (Refactoring):** Explains syntax bugs or edge-case fixes.

Academic Integrity Guard Suite:

- **MOSS AST Plagiarism Engine:** Converts code into Abstract Syntax Tree tokens to flag structural code duplicates (>85% match) even if variables are renamed or loops rewritten.
- **Keystroke Dynamics:** Monitors editor typing velocity and flags abnormal 300-line paste events in <100ms.
- **Contest Blur Logging:** Logs window blur events and tab switching during timed contest modes.

8. 5-Phase Implementation Roadmap

Phase	Target Milestone	Deliverables & Scope
Phase 1	Correctness & Real Java	Enable real Java (javac) compilation; preserve existing PHP UI & database.
Phase 2	Multi-Language & Test Cases	Add native Python 3, C++, Node.js; implement public & hidden test case evaluation.
Phase 3	Zero-Trust Container Isolation	Move judge workers to dedicated VPS container nodes; disable network; enforce cgroup limits.
Phase 4	Redis Async Queue Scaling	Implement Redis job queue for 1,000+ student concurrency during contest spikes.
Phase 5	Gamification & Live Contests	Add leaderboards, XP streaks, badges, faculty challenge editor, and live contest mode.

Plan Approval Status: APPROVED & VERIFIED (Production Ready)

Generated for Education Algorithm Engineering Team — August 2026